Compiler Design Lab

Date: 13-3-2026 **Name:** S Vishwajith
Assignment No.: 10 **Register No.:** 23BCE1145

Aim:

To implement a code to find Three Address code and represent it in quadruples and triples using C++.

Code:

```
#include <iostream>

#include <fstream>

#include <sstream>

#include <string>

#include <vector>

#include <map>

#include <set>

#include <stack>

#include <iomanip>

#include <cstdlib>

using namespace std;


enum TokKind

{

  T_ID,

  T_NUM,

  T_ASSIGN,

  T_PLUS,

  T_MINUS,
```

```
    T_STAR,
    T_SLASH,
    T_LT,
    T_GT,
    T_LE,
    T_GE,
    T_EQ,
    T_NEQ,
    T_AND,
    T_OR,
    T_NOT,
    T_LPAREN,
    T_RPAREN,
    T_LBRACKET,
    T_RBRACKET,
    T_LBRACE,
    T_RBRACE,
    T_SEMI,
    T_COMMA,
    T_IF,
    T_ELSE,
    T_WHILE,
    T_FOR,
    T_BREAK,
    T_EOF
};
```

struct *Token*

```
{
    TokKind kind;
    string val;
    int line;
};

class Lexer
{
    string src;
    size_t pos = 0;
    int line = 1;

    bool peek2(char a, char b) const
    {
        return pos + 1 < src.size() && src[pos] == a && src[pos + 1] == b;
    }

    void skipWS()
    {
        while (pos < src.size())
        {
            char c = src[pos];
            if (c == '\n')
            {
                ++line;
                ++pos;
            }
            else if (isspace((unsigned char)c))
```

```
      {
         ++pos;
      }
      else if (peek2('/', '/'))
      {
         while (pos < src.size() && src[pos] != '\n')
            ++pos;
      }
      else
         break;
   }
}

static TokKind keyword(const string &s)
{
   if (s == "if")
      return T_IF;
   if (s == "else")
      return T_ELSE;
   if (s == "while")
      return T_WHILE;
   if (s == "for")
      return T_FOR;
   if (s == "break")
      return T_BREAK;
   return T_ID;
}
```

```
public:
  explicit Lexer(string source) : src(move(source)) {}

  Token next()
  {
    skipWS();
    if (pos >= src.size())
      return {T_EOF, "", line};

    char c = src[pos];

    if (isdigit((unsigned char)c))
    {
      string num;
      while (pos < src.size() && isdigit((unsigned char)src[pos]))
        num += src[pos++];
      return {T_NUM, num, line};
    }

    if (isalpha((unsigned char)c) || c == '_')
    {
      string id;
      while (pos < src.size() &&
          (isalnum((unsigned char)src[pos]) || src[pos] == '_'))
        id += src[pos++];
      return {keyword(id), id, line};
    }
```

```
if (peek2('<', '='))
{
    pos += 2;
    return {T_LE, "<=", line};
}
if (peek2('>', '='))
{
    pos += 2;
    return {T_GE, ">=", line};
}
if (peek2('=', '='))
{
    pos += 2;
    return {T_EQ, "==", line};
}
if (peek2('!', '='))
{
    pos += 2;
    return {T_NEQ, "!=", line};
}
if (peek2('&', '&'))
{
    pos += 2;
    return {T_AND, "&&", line};
}
if (peek2('|', '|'))
{
    pos += 2;
```

```
    return {T_OR, "||", line};
}

++pos;
switch (c)
{
case '=':
    return {T_ASSIGN, "=", line};
case '+':
    return {T_PLUS, "+", line};
case '-':
    return {T_MINUS, "-", line};
case '*':
    return {T_STAR, "*", line};
case '/':
    return {T_SLASH, "/", line};
case '<':
    return {T_LT, "<", line};
case '>':
    return {T_GT, ">", line};
case '!':
    return {T_NOT, "!", line};
case '(':
    return {T_LPAREN, "(", line};
case ')':
    return {T_RPAREN, ")", line};
case '[':
    return {T_LBRACKET, "[", line};
```

```
    case ']':
        return {T_RBRACKET, "]", line};
    case '{':
        return {T_LBRACE, "{", line};
    case '}':
        return {T_RBRACE, "}", line};
    case ';':
        return {T_SEMI, ";", line};
    case ',':
        return {T_COMMA, ",", line};
    default:
        cerr << "Unknown character '" << c
            << "' at line " << line << '\n';
        return {T_EOF, "", line};
    }
}

vector<Token> tokenise()
{
    vector<Token> v;
    Token t;
    do
    {
        t = next();
        v.push_back(t);
    } while (t.kind != T_EOF);
    return v;
}
```

```
};

struct Instr
{
   string label;
   string op, arg1, arg2, result;

   string str() const
   {
      if (op == "goto")
         return "goto " + result;
      if (op == "if False")
         return "if False " + arg1 + " goto " + result;
      if (op == "=")
         return result + " = " + arg1;
      if (op == "[]r")
         return result + " = " + arg1 + "[" + arg2 + "]";
      if (op == "[]w")
         return arg1 + "[" + arg2 + "] = " + result;
      if (!arg2.empty())
         return result + " = " + arg1 + " " + op + " " + arg2;
      if (!arg1.empty())
         return result + " = " + op + " " + arg1;
      return op;
   }
};

class CodeGen
```

```
{
    vector<Token> toks;
    size_t cur = 0;
    vector<Instr> code;
    int tempCnt = 0;
    int labelCnt = 0;
    stack<string> breakTarget; // innermost loop exit label

    Token &peek() { return toks[cur]; }
    bool at(TokKind k) { return peek().kind == k; }
    Token consume() { return at(T_EOF) ? toks[cur] : toks[cur++]; }
    bool match(TokKind k)
    {
        if (at(k))
        {
            consume();
            return true;
        }
        return false;
    }

    void expect(TokKind k, const char *what)
    {
        if (!match(k))
        {
            cerr << "Parse error at line " << peek().line
                << ": expected " << what
                << ", got '" << peek().val << "'\n";
```

```
        exit(1);
    }
}

string freshTemp() { return "t" + to_string(++tempCnt); }
string freshLabel() { return "L" + to_string(++labelCnt); }

void emit(const string &op,
          const string &a1 = "",
          const string &a2 = "",
          const string &res = "")
{
    code.push_back({"", op, a1, a2, res});
}
void emitLabel(const string &lbl)
{
    code.push_back({lbl, "label", "", "", ""});
}

string parseExpr() { return parseOr(); }

string parseOr()
{
    string lhs = parseAnd();
    while (at(T_OR))
    {
        consume();
        string rhs = parseAnd();
```

```
        string t = freshTemp();
        emit("||", lhs, rhs, t);
        lhs = t;
    }
    return lhs;
}

string parseAnd()
{
    string lhs = parseRel();
    while (at(T_AND))
    {
        consume();
        string rhs = parseRel();
        string t = freshTemp();
        emit("&&", lhs, rhs, t);
        lhs = t;
    }
    return lhs;
}

string parseRel()
{
    string lhs = parseAdd();
    for (;;)
    {
        string op;
        switch (peek().kind)
```

```
    {
    case T_LT:
        op = "<";
        break;
    case T_GT:
        op = ">";
        break;
    case T_LE:
        op = "<=";
        break;
    case T_GE:
        op = ">=";
        break;
    case T_EQ:
        op = "==";
        break;
    case T_NEQ:
        op = "!=";
        break;
    default:
        return lhs;
    }
    consume();
    string rhs = parseAdd();
    string t = freshTemp();
    emit(op, lhs, rhs, t);
    lhs = t;
}
```

```
}

string parseAdd()
{
   string lhs = parseMul();
   while (at(T_PLUS) || at(T_MINUS))
   {
      string op = consume().val;
      string rhs = parseMul();
      string t = freshTemp();
      emit(op, lhs, rhs, t);
      lhs = t;
   }
   return lhs;
}

string parseMul()
{
   string lhs = parseUnary();
   while (at(T_STAR) || at(T_SLASH))
   {
      string op = consume().val;
      string rhs = parseUnary();
      string t = freshTemp();
      emit(op, lhs, rhs, t);
      lhs = t;
   }
   return lhs;
```

```
}

string parseUnary()
{
   if (at(T_MINUS))
   {
      consume();
      string t = freshTemp();
      emit("-", parsePrimary(), "", t);
      return t;
   }
   if (at(T_NOT))
   {
      consume();
      string t = freshTemp();
      emit("!", parsePrimary(), "", t);
      return t;
   }
   return parsePrimary();
}

string parsePrimary()
{
   if (at(T_LPAREN))
   {
      consume();
      string v = parseExpr();
      expect(T_RPAREN, ")");
```

```
    return v;
}
if (at(T_NUM))
    return consume().val;

if (at(T_ID))
{
    string name = consume().val;

    if (at(T_LBRACKET))
    {
        consume();
        string idx1 = parseExpr();
        expect(T_RBRACKET, "]");

        if (at(T_LBRACKET))
        {
            consume();
            string idx2 = parseExpr();
            expect(T_RBRACKET, "]");

            string tRow = freshTemp();
            string tOff = freshTemp();
            string tVal = freshTemp();
            emit("*", idx1, "cols", tRow);
            emit("+", tRow, idx2, tOff);
            emit("[]r", name, tOff, tVal);
            return tVal;
```

```
        }

        string t = freshTemp();
        emit("[]r", name, idx1, t);
        return t;
      }
      return name;
    }

    cerr << "Unexpected token '" << peek().val
       << "' at line " << peek().line << '\n';
    exit(1);
}

void parseBlock()
{
    if (at(T_LBRACE))
    {
      consume();
      while (!at(T_RBRACE) && !at(T_EOF))
        parseStmt();
      expect(T_RBRACE, "}");
    }
    else
    {
      parseStmt();
    }
}
```

```
void parseIf()
{
    consume();
    expect(T_LPAREN, "(");
    string cond = parseExpr();
    expect(T_RPAREN, ")");

    string lFalse = freshLabel();
    string lEnd = freshLabel();

    emit("if False", cond, "", lFalse);
    parseBlock();

    if (at(T_ELSE))
    {
        consume();
        emit("goto", "", "", lEnd);
        emitLabel(lFalse);
        if (at(T_IF))
            parseIf();
        else
            parseBlock();
        emitLabel(lEnd);
    }
    else
    {
        emitLabel(lFalse);
```

```
    }
}

void parseWhile()
{
    consume();

    string lStart = freshLabel();
    string lEnd = freshLabel();

    breakTarget.push(lEnd);

    emitLabel(lStart);
    expect(T_LPAREN, "(");
    string cond = parseExpr();
    expect(T_RPAREN, ")");

    emit("if False", cond, "", lEnd);
    parseBlock();
    emit("goto", "", "", lStart);
    emitLabel(lEnd);

    breakTarget.pop();
}

void parseFor()
{
    consume();
```

```
expect(T_LPAREN, "(");

if (!at(T_SEMI))
   parseAssign();
expect(T_SEMI, ";");

string lCond = freshLabel();
string lBody = freshLabel();
string lIncr = freshLabel();
string lEnd = freshLabel();

breakTarget.push(lEnd);

emitLabel(lCond);

string cond = "1";
if (!at(T_SEMI))
   cond = parseExpr();
expect(T_SEMI, ";");

emit("if False", cond, "", lEnd);
emit("goto", "", "", lBody);

size_t incrStart = cur;
{
   int depth = 1;
   while (cur < toks.size() && depth > 0)
   {
```

```
        if (toks[cur].kind == T_LPAREN)
          ++depth;
        if (toks[cur].kind == T_RPAREN)
          --depth;
        ++cur;
      }
    }

    emitLabel(lBody);
    parseBlock();

    emitLabel(lIncr);
    size_t afterBody = cur;
    cur = incrStart;
    if (!at(T_RPAREN))
      parseAssign();
    cur = afterBody;

    emit("goto", "", "", lCond);
    emitLabel(lEnd);

    breakTarget.pop();
  }

  void parseAssign()
  {
    if (!at(T_ID))
    {
```

```
    parseExpr();
    return;
}

string name = consume().val;

if (at(T_LBRACKET))
{
    consume();
    string idx1 = parseExpr();
    expect(T_RBRACKET, "]");

    if (at(T_LBRACKET))
    {
        consume();
        string idx2 = parseExpr();
        expect(T_RBRACKET, "]");
        expect(T_ASSIGN, "=");
        string rhs = parseExpr();

        string tRow = freshTemp();
        string tOff = freshTemp();
        emit("*", idx1, "cols", tRow);
        emit("+", tRow, idx2, tOff);
        emit("[]w", name, tOff, rhs);
        return;
    }
```

```
        expect(T_ASSIGN, "=");
        string rhs = parseExpr();
        emit("[]w", name, idx1, rhs);
        return;
    }

    if (at(T_ASSIGN))
    {
        consume();
        string rhs = parseExpr();
        emit("=", rhs, "", name);
        return;
    }

    --cur;
    parseExpr();
}

void parseStmt()
{
    switch (peek().kind)
    {
    case T_IF:
        parseIf();
        break;
    case T_WHILE:
        parseWhile();
        break;
```

```
      case T_FOR:
        parseFor();
        break;
      case T_BREAK:
        consume();
        if (!breakTarget.empty())
          emit("goto", "", "", breakTarget.top());
        else
          cerr << "Warning: 'break' outside loop at line "
              << peek().line << '\n';
        match(T_SEMI);
        break;
      case T_LBRACE:
        parseBlock();
        break;
      case T_SEMI:
        consume();
        break;
      default:
        parseAssign();
        match(T_SEMI);
        break;
      }
    }

public:
    explicit CodeGen(vector<Token> tokens) : toks(move(tokens)) {}
```

```
  void run()
  {
    while (!at(T_EOF))
      parseStmt();
  }

  const vector<Instr> &instructions() const { return code; }
};

static string pad(const string &s, int w)
{
  string r = s;
  if ((int)r.size() > w)
    r = r.substr(0, w);
  while ((int)r.size() < w)
    r += ' ';
  return r;
}

static void hRule(int style, const vector<int> &widths)
{
  const char *L[] = {"+", "+", "+"};
  const char *M[] = {"+", "+", "+"};
  const char *R[] = {"+", "+", "+"};
  cout << L[style];
  for (size_t i = 0; i < widths.size(); ++i)
  {
    for (int j = 0; j < widths[i] + 2; ++j)
```

```
        cout << "-";
      cout << (i + 1 < widths.size() ? M[style] : R[style]);
   }
   cout << '\n';
}

static void printTAC(const vector<Instr> &code)
{
   cout <<
"\n+==================================================================+\n"
        "|           THREE-ADDRESS CODE  (Labelled Form)          |\n"
        "+==================================================================+\
n\n";

   int seq = 0;
   for (const auto &ins : code)
   {
      if (ins.op == "label")
      {
         if (seq > 0)
            cout << '\n';
         cout << ins.label << ":\n";
      }
      else
      {
         cout << "  [" << setw(3) << setfill('0') << seq
             << setfill(' ') << "]   " << ins.str() << '\n';
         ++seq;
      }
```

```
    }
    cout << '\n';
}

static void printQuadruple(const vector<Instr> &code)
{
    cout <<
"\n+==================================================================+\n"
        "|                    QUADRUPLE TABLE                    |\n"
        "+==================================================================+\
n";

    const int WI = 4, WO = 10, WA = 14, WB = 14, WR = 16;
    hRule(0, {WI, WO, WA, WB, WR});
    cout << "| " << pad("#", WI) << " | "
        << pad("Op", WO) << " | "
        << pad("Arg1", WA) << " | "
        << pad("Arg2", WB) << " | "
        << pad("Result", WR) << " |\n";
    hRule(1, {WI, WO, WA, WB, WR});

    int n = 0;
    for (const auto &ins : code)
    {
        if (ins.op == "label")
            continue;
        cout << "| " << pad(to_string(n++), WI) << " | "
            << pad(ins.op, WO) << " | "
            << pad(ins.arg1, WA) << " | "
```

```
        << pad(ins.arg2, WB) << " | "
        << pad(ins.result, WR) << " |\n";
    }
    hRule(2, {WI, WO, WA, WB, WR});
    cout << '\n';
}

struct Triple
{
    string op, arg1, arg2;
};

static string resolveTriple(const string &arg,
                            const map<string, int> &t2i)
{
    auto it = t2i.find(arg);
    return (it != t2i.end()) ? "(" + to_string(it->second) + ")" : arg;
}

static void printTriple(const vector<Instr> &code)
{
    cout <<
"\n+==================================================================+\n"
        "|                    TRIPLE TABLE                    |\n"
        "+==================================================================+\
n";

    vector<Triple> triples;
    map<string, int> t2i;
```

```
for (const auto &ins : code)
{
    if (ins.op == "label")
        continue;

    Triple tr;
    tr.op = ins.op;

    if (ins.op == "=")
    {
        tr.arg1 = resolveTriple(ins.arg1, t2i);
        tr.arg2 = resolveTriple(ins.result, t2i);
    }
    else if (ins.op == "goto")
    {
        tr.arg1 = ins.result;
        tr.arg2 = "";
    }
    else if (ins.op == "if False")
    {
        tr.arg1 = resolveTriple(ins.arg1, t2i);
        tr.arg2 = ins.result;
    }
    else if (ins.op == "[]w")
    {
        tr.arg1 = ins.arg1 + "[" + resolveTriple(ins.arg2, t2i) + "]";
        tr.arg2 = resolveTriple(ins.result, t2i);
```

```
    }
    else if (ins.arg2.empty() && !ins.arg1.empty())
    {
        tr.arg1 = resolveTriple(ins.arg1, t2i);
        tr.arg2 = resolveTriple(ins.result, t2i);
    }
    else
    {
        tr.arg1 = resolveTriple(ins.arg1, t2i);
        tr.arg2 = resolveTriple(ins.arg2, t2i);
    }

    int idx = (int)triples.size();
    triples.push_back(tr);

    if (!ins.result.empty() && ins.result[0] == 't' && isdigit((unsigned char)ins.result[1]))
        t2i[ins.result] = idx;
}

const int WI = 4, WO = 10, WA = 18, WB = 18;
hRule(0, {WI, WO, WA, WB});
cout << "| " << pad("#", WI) << " | "
    << pad("Op", WO) << " | "
    << pad("Arg1", WA) << " | "
    << pad("Arg2", WB) << " |\n";
hRule(1, {WI, WO, WA, WB});

for (int i = 0; i < (int)triples.size(); ++i)
```

```
  {
    cout << "| " << pad(to_string(i), WI) << " | "
        << pad(triples[i].op, WO) << " | "
        << pad(triples[i].arg1, WA) << " | "
        << pad(triples[i].arg2, WB) << " |\n";
  }
  hRule(2, {WI, WO, WA, WB});
  cout << '\n';
}

int main(int argc, char *argv[])
{
  if (argc < 2)
  {
    cout << "Give the input file name next to the execution file." << endl;
    return 1;
  }

  ifstream fin(argv[1]);
  if (!fin)
  {
    cerr << "Error: cannot open '" << argv[1] << "'\n";
    return 1;
  }

  string src((istreambuf_iterator<char>(fin)),
          istreambuf_iterator<char>());
```

```
    cout << "\nSource file : " << argv[1] << "\n";
    cout << "-------------------------------------------------------------\n";
    cout << src;
    if (!src.empty() && src.back() != '\n')
        cout << '\n';
    cout << "-------------------------------------------------------------\n";

    Lexer lex(src);
    auto tokens = lex.tokenise();

    CodeGen gen(move(tokens));
    gen.run();
    const auto &code = gen.instructions();

    if (code.empty())
    {
        cerr << "\nNothing to generate -- check the input file for syntax errors.\n";
        return 1;
    }

    printTAC(code);
    printQuadruple(code);
    printTriple(code);

    return 0;
}
```

Inputs and Outputs:

Code 1:

```
PS C:\Users\vishw\Coding\Compiler-Lab\Week-11> .\TAC.exe input1.txt

Source file : input1.txt
----------------------------------------------------------------
i = 0
sum = 0
fail_count = 0
while (i < n)
{
    if (marks[i] > 50)
        sum = sum + marks[i]
    else
        fail_count = fail_count + 1
    i = i + 1
}
----------------------------------------------------------------
```

```
+==================================================================+
|              THREE-ADDRESS CODE  (Labelled Form)                 |
+==================================================================+

  [000]   i = 0
  [001]   sum = 0
  [002]   fail_count = 0

L1:
  [003]   t1 = i < n
  [004]   if False t1 goto L2
  [005]   t2 = marks[i]
  [006]   t3 = t2 > 50
  [007]   if False t3 goto L3
  [008]   t4 = marks[i]
  [009]   t5 = sum + t4
  [010]   sum = t5
  [011]   goto L4

L3:
  [012]   t6 = fail_count + 1
  [013]   fail_count = t6

L4:
  [014]   t7 = i + 1
  [015]   i = t7
  [016]   goto L1

L2:
```

QUADRUPLE TABLE

#	Op	Arg1	Arg2	Result
0	=	0		i
1	=	0		sum
2	=	0		fail_count
3	<	i	n	t1
4	if False	t1		L2
5	[]r	marks	i	t2
6	>	t2	50	t3
7	if False	t3		L3
8	[]r	marks	i	t4
9	+	sum	t4	t5
10	=	t5		sum
11	goto			L4
12	+	fail_count	1	t6
13	=	t6		fail_count
14	+	i	1	t7
15	=	t7		i
16	goto			L1

TRIPLE TABLE

#	Op	Arg1	Arg2
0	=	0	i
1	=	0	sum
2	=	0	fail_count
3	<	i	n
4	if False	(3)	L2
5	[]r	marks	i
6	>	(5)	50
7	if False	(6)	L3
8	[]r	marks	i
9	+	sum	(8)
10	=	(9)	sum
11	goto	L4	
12	+	fail_count	1
13	=	(12)	fail_count
14	+	i	1
15	=	(14)	i
16	goto	L1	

Code 2:

```
PS C:\Users\vishw\Coding\Compiler-Lab\Week-11> .\TAC.exe input2.txt

Source file : input2.txt
-------------------------------------------------------------
if (marks > 80 && attendance > 90)
    scholarship = 2
else if (marks > 70 || sports == 1)
    scholarship = 1
else
    scholarship = 0
-------------------------------------------------------------

+==============================================================+
|              THREE-ADDRESS CODE  (Labelled Form)             |
+==============================================================+

  [000]   t1 = marks > 80
  [001]   t2 = attendance > 90
  [002]   t3 = t1 && t2
  [003]   if False t3 goto L1
  [004]   scholarship = 2
  [005]   goto L2

L1:
  [006]   t4 = marks > 70
  [007]   t5 = sports == 1
  [008]   t6 = t4 || t5
  [009]   if False t6 goto L3
  [010]   scholarship = 1
  [011]   goto L4

L3:
  [012]   scholarship = 0

L4:

L2:
```

QUADRUPLE TABLE

#	Op	Arg1	Arg2	Result
0	>	marks	80	t1
1	>	attendance	90	t2
2	&&	t1	t2	t3
3	if False	t3		L1
4	=	2		scholarship
5	goto			L2
6	>	marks	70	t4
7	==	sports	1	t5
8	||	t4	t5	t6
9	if False	t6		L3
10	=	1		scholarship
11	goto			L4
12	=	0		scholarship

TRIPLE TABLE

#	Op	Arg1	Arg2
0	>	marks	80
1	>	attendance	90
2	&&	(0)	(1)
3	if False	(2)	L1
4	=	2	scholarship
5	goto	L2	
6	>	marks	70
7	==	sports	1
8	||	(6)	(7)
9	if False	(8)	L3
10	=	1	scholarship
11	goto	L4	
12	=	0	scholarship

Code 3:

```
PS C:\Users\vishw\Coding\Compiler-Lab\Week-11> .\TAC.exe input3.tx

Source file : input3.txt
----------------------------------------------------------------
i = 0
found = 0
while (i < n)
{
    if (A[i] == key)
    {
        found = 1
        break
    }
    i = i + 1
}
----------------------------------------------------------------

+================================================================+
|              THREE-ADDRESS CODE  (Labelled Form)               |
+================================================================+

  [000]   i = 0
  [001]   found = 0

L1:
  [002]   t1 = i < n
  [003]   if False t1 goto L2
  [004]   t2 = A[i]
  [005]   t3 = t2 == key
  [006]   if False t3 goto L3
  [007]   found = 1
  [008]   goto L2

L3:
  [009]   t4 = i + 1
  [010]   i = t4
  [011]   goto L1

L2:
```

QUADRUPLE TABLE

#	Op	Arg1	Arg2	Result
0	=	0		i
1	=	0		found
2	<	i	n	t1
3	if False	t1		L2
4	[]r	A	i	t2
5	==	t2	key	t3
6	if False	t3		L3
7	=	1		found
8	goto			L2
9	+	i	1	t4
10	=	t4		i
11	goto			L1

TRIPLE TABLE

#	Op	Arg1	Arg2
0	=	0	i
1	=	0	found
2	<	i	n
3	if False	(2)	L2
4	[]r	A	i
5	==	(4)	key
6	if False	(5)	L3
7	=	1	found
8	goto	L2	
9	+	i	1
10	=	(9)	i
11	goto	L1	

Result:

We have successfully implemented a C++ code to find Three Address Code and it's data structures (quadruple, triple).